Automatic Code Documentation with Syntax Trees and GPT
Alleviating Software Development's Most Redundant Task

Tyler Thomas Procko
Department of Electrical Engineering and Computer Science
Embry-Riddle Aeronautical University
Daytona Beach, United States of America
prockot@my.erau.edu

Steve Collins
ForumAPI
Hoboken, United States of America
steve.collins.jc@gmail.com

Context: The explicit documentation of written software has historically been a common task within the software development life cycle, providing developers, users and other stakeholders with human-readable artifacts alongside code. Code documentation is a tedious process plagued by a host of issues, most notably that of documentation debt, which can occur when code is modified and its documentation no longer reflects it. With the recent advancement of large language models (LLMs), e.g., OpenAI's GPT-3.5 and GPT-4, many common software development processes are being assisted by LLMs. For instance, GPT is used frequently by software developers as a know-it-all software consultant.
Objective: In this paper, we discuss the development and use of the Automatic Code Documentation with Compilers (ACDC) system for the C#/ NET language, which utilizes GPT-3.5 and abstract syntax trees (ASTs) generated from the Roslyn compiler. ACDC is a Continuous Integration and Continuous Delivery (CI/CD) system that is integrated with Git repositories for real-time generation of code documentation, alleviating the need to manually update documentation.
Method: The ACDC system is implemented prototypically and evaluated on documenting a small set of Roslyn source code. A survey of five experienced software developers provides an evaluation of the system.
Results: ACDC is shown to be capable of producing good code documentation. As indicated by the survey, GPT-3.5 was preferred by all developers, because of its terseness, where GPT-4 was not well-liked as a documentation generator, because of its wordiness.
Conclusion: The development of the ACDC system has occurred at a precipitous and frenetic time in the history of computation. In the months and years to come, there is no doubt that large-scale AI will expand in use. ACDC presents a theoretically grounded system that mitigates some of the stochastic uncertainty of using GPT through ASTs and well-established prompt engineering techniques to automate the process of code documentation for software practitioners.

***Keywords*—code documentation, CI/CD, DevOps, GPT, syntax trees, ASTs, SDLC, software process improvement**

1. Introduction

As a discipline, software engineering has been extant since at least the early 1970s [1]. Attendant with software engineering is the Software Development Life Cycle (SDLC). The SDLC has been a subject of heated discussion and varying implementations for software developers since the 1980s, and has seen several major zeitgeists, e.g., with the early documentation-focused Waterfall and Waterfall-like methods, and the more recent Agile methods, e.g., Scrum and Extreme Programming [2]. Regardless of the SDLC model employed, documentation invariably exists alongside code, either for the code developers or the end users of the code. Code documentation can be as simple as comments in source code or enterprise-ready, as published documentation sites on the Web.

Deriving from the Agile mindset, a practice adopted commonly in contemporary software engineering is Continuous Integration and Continuous Delivery (CI/CD), which is intended to improve software turnaround time by introducing automation to certain stages of the SDLC [3, 4]. The CI/CD practice is often described as a *pipeline*, where particular stretches of the pipeline, such as testing, can be completely automated, allowing developers to focus on principal tasks, e.g., development itself. Much of the CI/CD community is open-source; various tools exist to allow automation at various stages of the SDLC, e.g., the popular Jenkins [5] and Travis CI [6]. CI/CD pipeline automation tools often focus on the automation of test suites, static code analysis and deployment protocols; but the automation of code documentation within a CI/CD pipeline remains a novel idea.

Code documentation has long been established as an annoying part of the SDLC to software developers, who often express a lack of motivation to document code because it is tedious, difficult, interrupts actual coding progress and interferes with time constraints, e.g., client deliverables [7]. It is widely accepted that programmers are "notorious" for neglecting source code documentation, given the considerations just described [8]. Additionally, it happens all too frequently that, as source code changes, its documentation becomes outdated and irrelevant [9]. This phenomenon, documentation debt, is a direct result of the need to maintain documentation alongside code, where the code inevitably takes precedence and outpaces its documentation, causing it to be incomplete or even missing [10]. To better allow CI/CD, it would be ideal to alleviate this developer pain with an automated documentation platform.

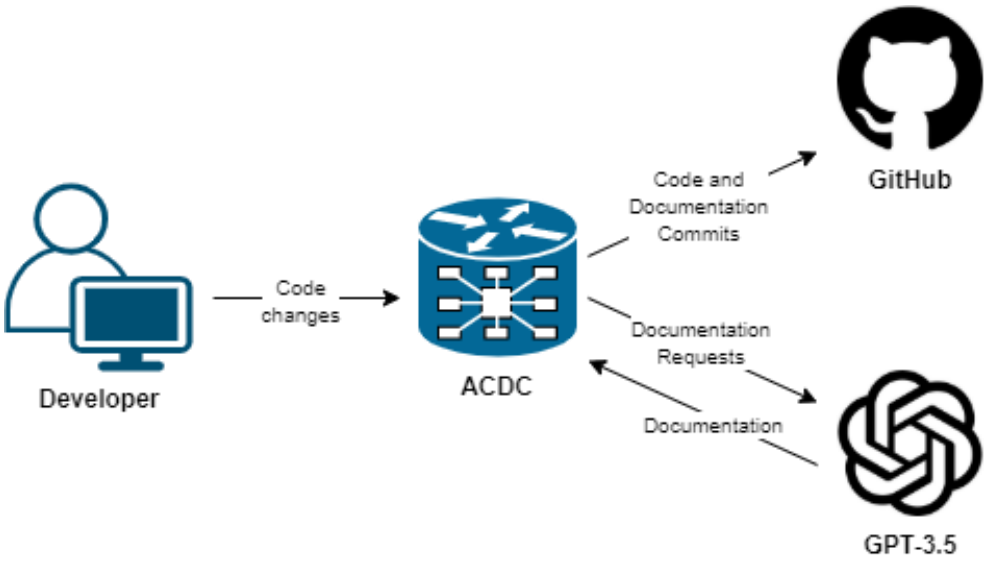

Figure 1. The simplified architecture of the ACDC system.

Various Deep Learning (DL) approaches have been used to summarize text. The general task of text summarization, not specific to code, using pretrained, open-source LLMs, has been well researched and proven to be one of the Transformer's most successful applications, achieving state-of-the-art (SOTA) across three common datasets [11]. Automatic *code documentation* is not an explicitly published topic, but automatic *code summarization* (CS) is. Alongside CS is code generation (CG). Research on tandem training of CS and CG models as a dual-task process has been conducted, with both CS and CG models surpassing SOTA models [12]. CS and CG are both important tasks in automatic code documentation, because, for example, the documentation of methods requires summarizing them into natural language statements for readers, and good code documentation often includes examples of method invocations, which must be derived from some CG model.

This paper frames code documentation as an entirely automatable CI/CD process. To this end, we present a system, Automatic Code Documentation with Compilers (ACDC), that utilizes abstract syntax trees generated from the Roslyn C#/.NET compiler and GPT-3.5, integrated with the GitHub workflow for real-time code documentation. The structure of this paper is as follows:

- Section II: background information on the history of automatic code documentation, LLM prompting and compilers
- Section III: this work's motivation with respect to the identified problem
- Section IV: the proposed ACDC system
- Section V: survey-based evaluation and analysis of the ACDC system
- Section VI: works related to automatic code documentation
- Section VII: discussion about ACDC, its extension to other languages and other facets of the SDLC; also, future work
- Section VIII: concluding remarks, including forward-looking statements

2. Background

This section presents some necessary background information regarding the historical precedence of automatic code documentation, compilers, abstract syntax trees and LLM prompting techniques, which are necessary for a proper understanding of the sections to follow.

2.1 Automatic Code Documentation

Speaking historically, the automatic generation of code documentation began around 2010 with the generation of summary comments for class methods, e.g., in Java [13]. This addressed a useful, albeit relatively small, facet of the documentation process, potentially helping reduce developer workload. Shortly after, the automatic generation of natural language summaries for entire Java classes along with methods was proposed, building off the previous work for summarizing individual methods [14, 15]. These earlier means of automatically generating code summaries were simplistic but laid the groundwork for later techniques.

In 2014, McBurney and McMillan improved upon these papers by considering method *context*, which includes the dependencies of the method and its place in the project hierarchy [16, 9]. This improved work utilized the Software Word Usage Model (SWUM), a means of representing code statements in terms of their parts of speech (e.g., nouns, verbs, etc.), to

extract lexical relationships from source code. SWUM was proposed as part of a dissertation thesis in 2010 [17]. In 2017, McBurney et al. proposed an automated system that employs static source code analysis with off-the-shelf tools and textual feature extraction to derive source code metrics, e.g., lines of code, class complexity, method cohesion, inheritance depth, etc., to feed into a model to identify which classes are important, to help programmers prioritize their documentation tasks [8].

In 2017, there began a transitional period that profoundly impacted the progress of automatic code summarization: the groundbreaking neural network architecture, the Transformer, was introduced, which utilizes a novel self-attention mechanism, allowing it to outperform both Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) [18]. With the introduction of the Transformer architecture, powerful, pre-trained LLMs with billions of parameters were introduced in the period of 2018-2023. Some of the more popular LLMs are given below:

- OpenAI's (Generative Pre-trained from Transformers) GPT [19, 20, 21]
- Google's Fine-tuned LAnguage Net (FLAN) and Pathways Language Model (PaLM) [22] [23]
- Bidirectional Encoder Representations from Transformers (BERT) [24]
- Facebook's (Meta's) Bidirectional and Auto-Regressive Transformers (BART) [25]
- Facebook's (Meta's) Large Language Model Meta AI (LLaMA) [26]

After the release of the transformer architecture, well-established, empirical evaluation of LLMs trained on code began around 2020 [27]. A Transformer-based code completion tool, IntelliCode Compose, capable of real-time incorporation with common integrated development environments (IDEs), e.g., Visual Studio Code, showed tremendous capability for cognizant developers who utilized it as a digital programming assistant for a variety of tasks [28]. OpenAI released the Codex LLM in 2021 [29]. Codex powers GitHub's Copilot, described as an "AI pair programmer", that can integrate with certain IDEs, e.g., Visual Studio Code, providing suggestions while coding and acting as a software consultant when needed. GitHub Copilot has been evaluated as being useful for simpler software development tasks and as a "seed" coder, where the human developer is needed to combine, extract, synthesize and otherwise finalize the output into properly functional code, e.g., because LLMs trained for code often generate code that relies on undefined helper functions [30, 31]. While GitHub Copilot is a useful tool for software developers, the quality of its generated code depends heavily on the conciseness and correctness of the natural language prompts provided by the developer [32]. In 2022, OpenAI deprecated the Codex model, because GPT-3 and GPT-3.5 (ChatGPT) performed exceptionally well at coding tasks, as well as various other tasks, granting users the ability to "chat" with the models in a manner much more amenable to workflow progression.

GPT-4 was released for public use in early 2023. Building on GPT-3.5, GPT-4 has advanced reasoning capabilities and increased attention, in addition to having the distinction as being the first multimodal model in mass use, as it can accept image and text inputs, responding in text [33]. While GPT-4 is superior to GPT-3.5 in almost all evaluations [34], for real-time use in large systems, it is not currently ideal because of its lack of speed and higher price compared to GPT-3.5. As such, in this proposal, GPT-3.5 is used, and GPT-4 is only considered for future work.

2.2. Effective LLM Prompting

The atomic process for LLM content generation is to prompt and to receive a response. Despite this perceived simplicity, there are a variety of empirically evaluated techniques to push the response of prompted LLMs in a desired direction. This field of study, prompt engineering or prompt programming, came about in 2021, with evaluations on GPT-3 [35] and text-to-image generative models [36]. Prompt engineering content generation can be thought of as *programming in natural language*. Because refactoring a prompt into a syntactically different, albeit semantically equivalent, prompt can drastically change the response of an LLM, some have asked the question: is there a replicable technique for making prompts to get a desired behavior? An answer to this question is not necessarily answerable definitively, because of the ambiguous nature of natural language.

Regardless, a common observation that has evolved into a prompting technique is that of 1- or few-shot prompting, where 1 or more examples are provided in the prompt; this technique consistently improves the output of LLMs, e.g., GPT, assuming the examples are distinguished from the prompt itself, and do not contaminate its semantics. The study of language has been applied to LLM prompting, with many techniques of human discourse materializing in prompt engineering techniques. The authors of these techniques state that they should be considered as reinforcing techniques, intended to shift a model toward a desired output; and, as such, they can be applied together to redundantly reinforce the LLM [35].

Some have brought prompt engineering into the domain of software engineering, applying it to common SDLC activities, e.g., improving code quality, refactoring code, eliciting software requirements and designing software [37, 38]. From these works, a prompt pattern form has been proposed, structured with the following facets: name and classification, intent, motivation, structure and participants, example, and consequences. Six prompt categories (classifications) have been identified: input semantics, output customization, error identification, prompt improvement, interaction and context control [37]. These general categories have been extended into the domain of software engineering, with the categories: requirements elicitation, system design and simulation, code quality, and refactoring, with each category having its respective prompt patterns, e.g., the

Table 1. General prompt engineering techniques.

Prompt Engineering Technique	**Definition**	**Example**
Direct task specification	A 0-shot (no example) prompt telling the model to do a task it knows; give the model *what*, with no *how*	*"Translate X into Y."*
Task specification by demonstration	A 1- or few-shot prompt telling the model to do something, given a demonstration; give the model *what* and *how*	*"Restructure this sentence X to adhere to this format Y."*
Task specification by memetic proxy	Using an analogy or proxy, such as a character or situation for open-ended tasks	*"As Ayn Rand, tell me your thoughts on generative AI"*, or *"You are my teacher and I am your student: explain generative AI to me in simple terms"*
Constraining behavior	Constraining the possible interpretations of the model by writing more complete prompts	*"Translate the following phrase from Italian to English: 'ciao amico' "* is a more constrained (and thus more predictable) prompt than *"Translate to English: ciao amico"*
Reasoning and metaprompt programming	For closed-ended questions, e.g., math questions; extending reasoning through multiple prompts, prompting the model to break down the answer into steps and dynamically responding to its intermediate answers	*"The problem asks us to do X. Let's consider each possible answer by splitting them into steps."*

requirements elicitation category has prompt patterns for simulating requirements, disambiguating specifications and simulating change requests [38].

Despite the boom in research utilizing LLMs for various tasks, concerns such as hallucination, or a model's tendency to confidently make up a response, are not fully understood [39]. LLMs have been referred to as "stochastic parrots", in that they, in the most reduced thinking possible, simply parrot the most probable or amenable response to a given prompt [40]. Regardless of these concerns, LLMs are useful tools for a plethora of tasks, given appropriate input and constraints. The authors of the software engineering prompt patterns note two important considerations for future work attempting to automate software engineering activities with LLMs [38]:

1. The true capability of LLMs, e.g., GPT-3.5, are not fully known
2. Human involvement is still needed to automate tasks, e.g., those in software engineering

In this proposal, the use of a LLM like GPT-3.5 for automating software engineering tasks is not viewed with much skepticism, because of the use of generated abstract syntax trees for source code, which provide serialized, standardized, contextualized datasets for source code elements of focus, e.g., methods and classes that need to be documented. In other words, prompting is only one piece of the proposed automatic code documentation system, where the inherent natural language ambiguity of prompting is reduced by using deterministic compiler-derived code representations, a contextualized prompting technique and well-established prompt patterns to constrain the behavior of the LLM.

2.3 Compilers and Abstract Syntax Trees

Because of the inherent ambiguity of natural language prompting, it is necessary to attempt to reduce the unpredictability of LLM responses by providing the model only with code in a standardized representation format. The most obvious means of doing so is by using a compiler.

Compilers translate programmed source code into a target language, e.g., assembly or machine code, to create an executable program. Aside from this primary directive, compilers have a host of other uses, including error checking, code optimization and static analysis, among others [41]. After lexical analysis, or tokenization of the source code, and during the parsing phase of compilation, a compiler will generate a parse tree (or concrete syntax tree); parse trees are often transformed into abstract syntax trees (ASTs), which are simplified, purely semantic representations of source code used in subsequent steps, including semantic analysis and intermediate language generation.

ASTs provide a common, simplified lingua franca, devoid of individual programming preferences and inessential delimiters, e.g., braces and semicolons. In this way, the use of an AST for ACD is prudent, because it is a direct means of constraining the LLM behavior, a well-established prompt engineering technique described in Table. ASTs derived and serialized from source code present an ideal source dataset for providing examples to the LLM, allowing 1- or few-shot prompting with examples in a common format, another recognized prompt engineering technique.

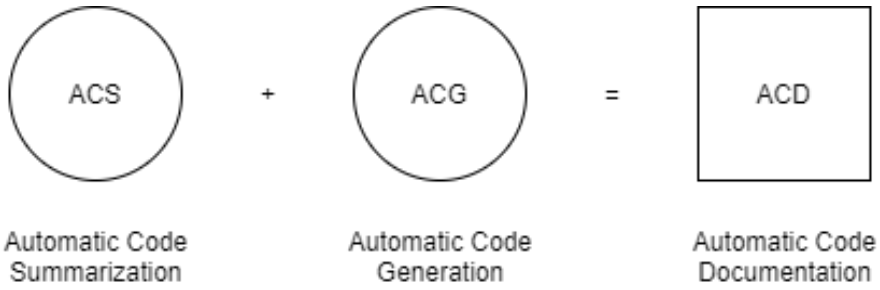

Figure 2. The combinatorial innovative nature of ACD.

This proposal is specific to the .NET ecosystem, so compilers of the C# and Visual Basic languages were considered. Microsoft's open-source .NET compiler platform, Roslyn, was therefore chosen because of its built-in suite of rich code analysis APIs, including a syntax visualizer as part of the SDK that generates browsable ASTs [42, 43].

3. Problem and Motivation

As already established, code documentation has long been an annoyance or impedance to software developers, and it is plagued by the issue of documentation debt, which occurs when changing code no longer matches its documentation [7, 8, 9, 10]. The framing of code documentation as a CI/CD task, to be automated with the appropriate use of LLMs and ASTs, would therefore be highly beneficial to the SDLC and to software developers in general.

It is apparent, from reviewing the scientific research landscape, that the two fields of interest, Automatic Code Summarization (ACS) and Automatic Code Generation (ACG), are well-researched areas with consistent publications. Automatic Code Documentation (ACD) is, however, a subject of relative obscurity and underrepresentation within the scientific community. It is posited that this is because the pieces needed for ACD are extant, it just remains for someone to put the proper pieces together to make ACD a reality. Incipient frameworks using GPT-3 for producing documentation of code methods (commenting) have been proposed [44], but there has not been a clear integration with a real-time CI/CD pipeline that can produce full-suite documentation. The Automatic Code Documentation with Compilers (ACDC) system described in this paper is a result of the appropriate combination of extant digital technologies, e.g., GPT-3.5, Octokit and Roslyn, into a CI/CD pipeline. In sociological terms, this act is generally termed as *combinatorial innovation* or *combinatorial breakthrough* [45]. Given the recent, meteoric rise of OpenAI's GPT series, principally the GPT-3.5 and GPT-4 models, the final piece for ACD fell into place, allowing the proposed system to be developed.

Granting the arrival of the GPT-3.5 and GPT-4 models, it is not unrealistic to envision the coming path for software development in the years ahead: common facets and practices within software engineering will be automated, or semi-automated, with the intelligent integration of powerful ML constructs, allowing developers to focus on actual development effort, improving their delivery potential and enriching the SDLC from end to end. The achievement of automatic code documentation with the proposed ACDC system is one paver stone in the path to come.

As such, the aim of this proposal is to fulfill the following goals:

1. Provide a framework for the development of a GPT-enabled SDLC ecosystem, by grounding on the process of software documentation as a totally automatable CI/CD task
2. Advance the empirical "trust" in GPT as a tool for the SDLC, by calling for the use of ASTs alongside source code in example prompts
3. Engender discussion about, and bootstrap future research in, automating facets of the SDLC with GPT

This work is intended to act as a springboard for furthering research into SDLC automation with LLMs by grounding on the widespread annoyance for software practitioners, to wit, software documentation. In the further sections of this paper, this aim is addressed.

4. Proposal

The proposed system is implemented as an API in the C# language, built with the Roslyn compiler, OpenAI's API to access GPT, and code specific to the problem. However, ACDC did not begin as such; in fact, it started as an extremely simplistic implementation in Python, to answer the question: *can you use GPT to automatically document the methods of a class*? It was this nascent question that began the work of this paper.

4.1. ACDC Prototype

Because of the expansive idea behind ACDC, its full implementation was not prudent; a simple prototype was therefore desired. The preliminary prototype of the system involved generating 10 C# methods to perform basic math operations, e.g., add,

Table 2. Prototype OpenAI API settings.

Parameter	Value	Justification
model	text-davinci-003	Quickest, cheapest and devoid of chat models' "system" messages
max_tokens	1024	Ensured completion response could fit reasonably (maximum is 4096)
temperature	0	Desired a deterministic baseline
All other settings	Default	Not needed for preliminary test

subtract, etc., through GPT-3.5. This validated the power of GPT-3.5 for generating code, and acted as proof for the coming requests for documentation. These methods were put into a C# class and extracted with simple Python code using regular expressions (regexes), then sent, along with a prompt requesting documentation in Markdown format, to the OpenAI API, using the most powerful Completion model, *text-davinci-003*.

This Completion model was used, as opposed to the Chat model *gpt-3.5-turbo*, because the Chat models invariably add perfunctory "social" prefixes and suffixes to every response, sometimes, even when directed not to. These are sometimes referred to as "system" messages [46]. For example, when asking GPT-3.5 to provide a good book to read on the subject of epistemology, the model will prefix its response with "*Certainly! Here's a book discussing epistemology:*", then provide the title of the relevant book, e.g., David Hume's *Enquiry*, and append its response with a summary statement, e.g., "*This book is a classic in the field of epistemology. It is important to remember that there are many other relevant books, and this one is just a starting point. Please let me know if I can help with anything else.*". For the purposes of the prototype, it was not desirous to have ancillary information; what was desired was the raw, explicit documentation of the prompted method. To this end, the Completion model, *text-davinci-003*, was used, because Completion models do not add ancillary information.

The used prompt is given here:

> *Document the given function Markdown style, following this format:*
> *1. Name the function with an H1 heading (1 #).*
> *2. Write a one-line, textual description of what the function does.*
> *3. Make a table for input/output parameters, with the table headings Parameter, Type and Description.*
> *4. Give two examples of function use inside Markdown code blocks (```)*
> *Here is the method:\n*

Where the regex-extracted method was appended to the input string after the newline delimiter (\n). The responses were quick, accurate, and included readable tables containing each method's signature, a textual description of the method and two examples of how to invoke it. This method is not scalable, however. The source code files of programming languages cannot be reliably parsed using regexes, because regexes are Chomsky type 3 (regular) grammars and most structured languages, e.g., those used in programming, are Chomsky type 2 (context-free) grammars, and since context-free grammars are more complex than regular grammars, it is not feasible to try to parse source code (beyond a limited subset, e.g., as that just described which only considers method bodies delimited by curly brackets {}) with regular expressions. So, it was posited that the use of a compiler to derive source code elements as needed was the appropriate remedy. As discussed earlier, the open-source Roslyn compiler was chosen for the C# language because of its plethora of code analysis APIs.

4.2 ACDC Implementation

Code summarization is a key element of ACDC. The summarization of code has been empirically evaluated with benchmark datasets and BLEU scoring with an LLM based on the GPT architecture as a task of relative simplicity [47]. Code summarization using a combination of GPT-2 and CodeBERT demonstrated high accuracy for the Java and Python languages [48]. With the more recent and powerful LLMs available, e.g., GPT-3.5 and GPT-4, the accuracy of code summarization is exceptional. In the previous section, it was discussed how the Chat models add social statements to the beginning and end of responses, e.g., "*Certainly! Here is…*" and "*Let me know if you require anything else!*". Accordingly, the Completion model, *text-davinci-003*, was used instead, to avoid this unnecessary text. In June of 2023, OpenAI released updates to the GPT-3.5 and GPT-4 models, notably, more "steerability" in terms of system messages [46], e.g., those "social" prefixes/suffices described in the previous section. So, instead of using the simple *text-davinci-003* Completion model, the Chat model, *gpt-3.5-turbo-0613*, was used instead. In some cases, the *gpt-3.5-turbo-16k-0613* model was used, because it has 4x the context length as the other just mentioned, which is useful for longer methods. The models can be directed with greater force with additional statements, by writing prompts that more tightly constrain model behavior.

An obstacle to the advancement of the ACDC system is the current limitation of GPT context length, or token length. For a given model, OpenAI prescribes a maximum number of tokens, which are calculated from the total number of characters / words (a process called encoding), including prompts sent to, and replies received from, the queried GPT model. The *max-_tokens* parameter for an API call to a Chat model dictates the allowable token length of the response itself, ignoring the input text. So, given a model with some maximum context length, e.g., 4k, the maximum allowable response length (in tokens) can

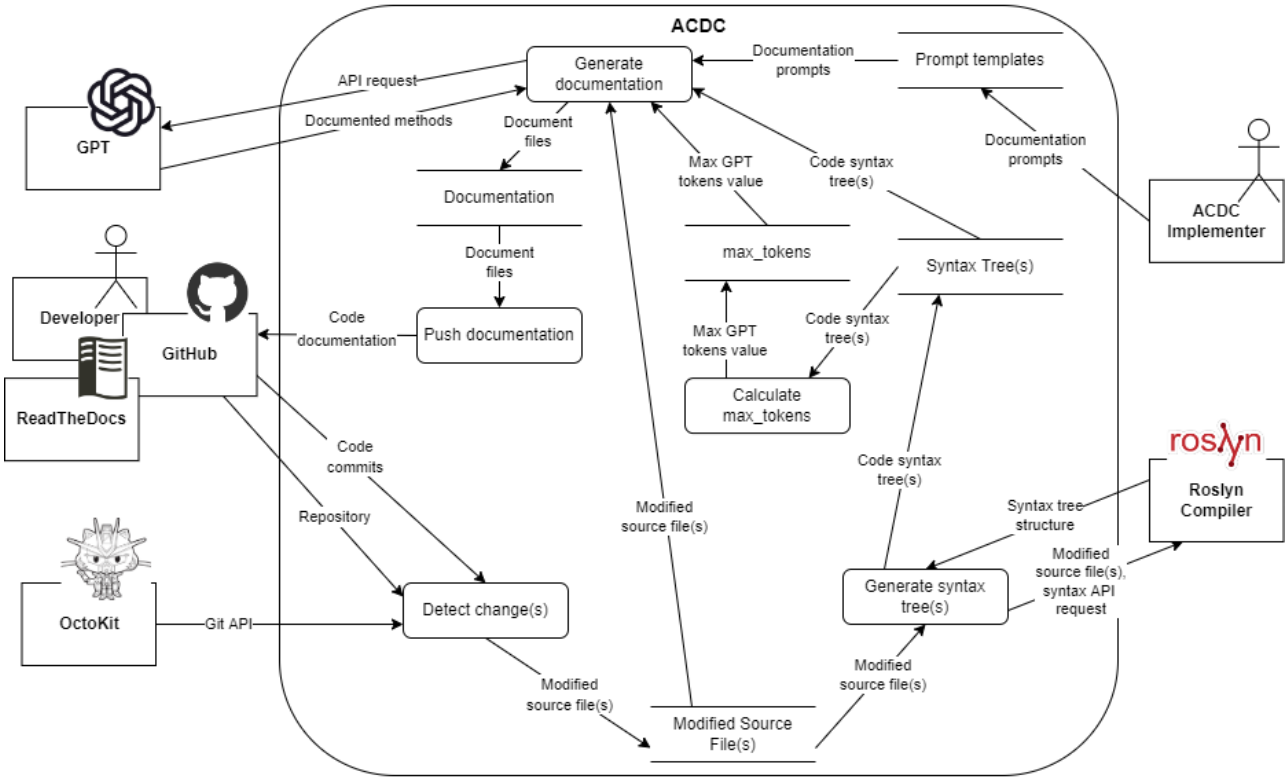

Figure 3. Data flow diagram of the proposed ACDC system.

be had by subtracting the input text token length from the maximum context length. Input length often varies, e.g., as method bodies can range from a few lines to several dozen lines. The *max_tokens* parameter defaults to infinite, which, presumably, means GPT responses will, if needed, take up the maximum allowable length. But this is not optimal, as use of the API can become costly with longer responses, so prudence is needed in determining the appropriate *max_tokens* value. Encoding varies for different models. For instance, the GPT-3.5 and GPT-4 models use the *cl100k_base* encoding. OpenAI has the tiktoken Python library to tokenize text and derive token lengths [49]. Utilizing this library can provide the upper limit of maximum allowable tokens, where the actual value of the *max_tokens* parameter must be derived from business concerns or project goals.

The framing of ACDC as a CI/CD pipeline is literal: its intent is to entirely automate the process of documentation, by removing the developer from the documentation lifecycle, allowing him to expend energy and time only in active development, where ACDC and its associated components generate code documentation in real time. This would effectively eliminate documentation debt, allowing guaranteed parity between code and documentation, which is indubitably appealing to software practitioners engaged with large codebases. This CI/CD data flow is given in Figure 3.

The Octokit .NET client library provides access to the GitHub API [50], allowing the ACDC system to interact programmatically with the associated source code repository where code is desired to be documented. In Figure 3. Data flow diagram of the proposed ACDC system. 3, the process labelled “generate documentation” is the most complex task of ACDC. It involves the ingestion of the individual code elements to be documented, e.g, methods, along with rules for the granularity of element ingestion, e.g., public methods only, the actual documentation request prompt to GPT-3.5, a calculated *max_tokens* value and the relevant AST context elements that will provide GPT-3.5 with enough context to properly document the method. This is the most customizable aspect of the ACDC system, and provides the developer with great freedom in the selection of things to be documented.

A core task of the ACDC system is the extraction of methods to be documented. In the naïve, preliminary tests, methods were extracted and documented without any *context*, i.e., the manner in which methods are invoked. A method’s context includes the statement calling it, the statement(s) giving it input and the statement(s) using its output [16]. In early 2014 tests, including the context of methods for automatic documentation using NLG algorithms was shown to generate better documentation than methods documented without context [16]. It is this finding that drives an important facet of the ACDC system, documentation contextualization. Documentation contextualization rests atop the core elements of the given programming language; in the case of this paper, the C#/.NET framework. The atomic code elements of this development framework are given below.

Algorithm 1 Pseudocode algorithm for the "generate documentation" process of ACDC
Input: Source files to be documented, documentation request prompt, code syntax tree(s), max GPT token value
Output: Documentation file(s)
Assumptions: Using the gpt-3.5-turbo-0613 Chat model

```
1.  Get a collection of modified source files to be documented
2.  Generate an AST for each source file using the Roslyn API
3.  while collection of source files not fully traversed do
4.    Extract code elements to be documented, e.g., methods, using attendant business logic (e.g., public methods only)
5.    Create a temporary documentation file (e.g., Markdown format) to correspond with a given source file
6.    while collection of code elements not fully traversed do
7.      Extract syntax tree context (e.g., namespaces, classes, etc.) relevant to the code element to be documented
8.      Combine the code element, syntax tree context and the documentation request prompt into a single string
9.      Calculate token length of code element for the combined GPT prompt string
10.     Calculate the maximum token length based on the previous step
11.     Set the max_tokens GPT parameter lower than the value in the previous step, but enough for GPT response based on business/project goals
12.     Send an API request for documentation through the GPT API
13.     Append the response to the documentation file
14.   end
15.   Save the temporary documentation file over the nominal one
16. end
```

<u>Canonical Code Elements in the C#/.NET Framework</u>

- Metadata
- Namespaces
  - Usings
    - Namespace
    - Alias
  - Classes
    - Name
    - Base Class
    - Implemented Interfaces
    - Methods, Functions, Constructors, Destructors, Operators
      - Name
      - Signature
      - Body
    - Properties
      - Name
      - Type
      - Getter
      - Setter
    - Nested Classes

The contextualization of methods to be documented is the most technically complex facet of ACDC, but it is made possible through the use of ASTs. ASTs allow the method to be extracted, along with any relevant context, determined by the developer. For example, within ACDC, a developer may specify that, for some method, he wishes to also include the usings of the class it is within and the class name it is within, all of which is fed into a single string to the GPT-3.5 API, appended to the structured prompt requesting documentation. This addition of context is posited to increase the accuracy of the LLM's output. If such documentation is insufficient, the developer may then include more method context from the AST, e.g., its place in the class hierarchy, the bodies of other methods within its class, methods it references in other classes, etc.

Building atop the documentation request prompt used in the ACDC prototype section of earlier, the documentation request prompt in the fully realized implementation of ACDC (referenced in step 8 of Algorithm 1) is more complex, and below, in Table 3, decomposed into its constituent prompt elements. This prompt utilizes the behavior constraint prompt engineering technique defined in Table 1, in addition to the general prompting technique of reinforcement, by telling it at the beginning and end of the prompt to not include system messages, resulting in more consistent output. It should be noted that, as indicated in Figure 3, the ACDC system views the use of GPT, current and future, as a problem of evolving optimization, so prompt "templates" are implemented, stored literally as strings, that allow for quick hot-swapping and rapid testing of GPT-generated code documentation when experimenting with different GPT models. As opposed to the Completion model used in the prototype of ACDC, the Chat models require a *messages* array as input, where each value of the array of a key/value pair, and

Table 3. Documentation request prompt (all rows are appended into one single string).

Prompt Preface	You are a software engineer experienced in code documentation. You do not give system messages, only documentation.
Transition	Here is the context of the code:\n
AST Context	[Add AST context as desired]
Transition	\nHere is the code to be documented:\n
Code Element	[Add the code element to be documented]
Transition	\nFor the given code, please provide the following in Markdown format:\n
Request 1	1. Name the function with an H1 heading\n
Request 2	2. Write a one-line description of what the function does, and include any info about its context to other classes\n
Request 3	3. Make a table for input/output parameters, with table headings Parameter, Type and Description\n
Request 4	4. Give two examples of use inside Markdown code blocks (```) \n
Transition	That is the end of the documentation request.
Reinforcement	Please do not include system messages.

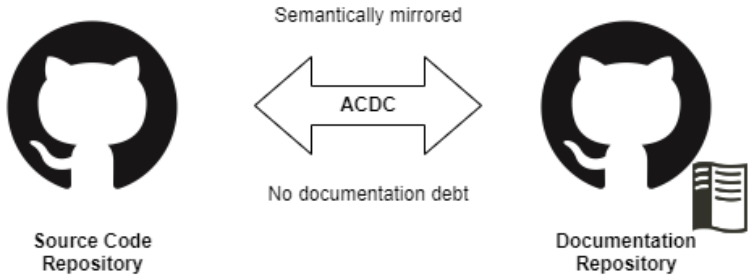

Figure 4. Source code and its documentation: automatically mirrored with ACDC.

each key must of one of *system*, *user*, *assistant*, or *function*. In ACDC, the array is only filled with a *user* key/value pair, where the value of the pair is the prompt from Table 3 as a string.

4.3 Documentation Platform

Insofar as the inevitable goal of the ACDC system is to output full-suite documentation for a given codebase, it is necessary to investigate documentation publication platforms that can make end-user viewing more amenable and accessible. Technically, because ACDC outputs documentation in Markdown format, these files could be indexed locally in any Web browser, but certain documentation publishing platforms include Web hosting, granular documentation organization, custom themes and other features, that are "nice to haves" for an end-user documentation view. Table 1 contains summaries of the considered documentation platforms. Some online development ecosystems (e.g., Confluence) allow the creation of wikis or documentation sites, but these are generally not meant for public access, and instead for internal teams, so they are not considered here.

Given some repository of source code, it would be ideal to have a sibling repository that contains the documentation of the source code in the other repository. In this sense, the most straightforward means of implementing this is through ReadTheDocs, which integrates seamlessly with GitHub; Sphinx or MkDocs could be used, although Sphinx is simpler to start with. Since

Table 4. Documentation platforms.

Platform	**Type**	**Format**	**Pros**	**Cons**
ReadTheDocs Community	Full suite	See Sphinx/MkDocs	Integrates with GitHub; free hosting	Public repos only
ReadTheDocs for Business	Full suite	See Sphinx/MkDocs	Public/private repos; more CPU	-
Sphinx	Document/static site generator	reStructuredText or MyST	Document cross-referencing	Do-it-yourself
MkDocs	Document/static site generator	reStructuredText	Plain Markdown and YAML file to host on Github	Do-it-yourself
Gitbook	Full suite	Proprietary, visual, Markdown	Streamlined, enterprise solution	Cost
GitHub pages	Static site host	HTML, Markdown	Simplest solution	Do-it-yourself
Wiki.js	Full suite	Markdown	Streamlined	No hosting
Docsie	Full suite	Markdown extended	Version control	Cost
Daux.io	Document/static site generator	Markdown	Simple	Do-it-yourself
Doxygen	Document/static site generator	Markdown, Javadoc-style code comments	Automatic code documentation extraction from annotated source code	Complex
Apiary	Full suite API platform	Proprietary, Markdown	-	Cost
SwaggerHub	Full suite API platform	Proprietary, JSON, YAML	-	Cost
Postman	Full suite API platform	Proprietary, Markdown, JSON	-	Cost

ReadTheDocs is the simplest documentation service supporting Markdown documentation, that additionally integrates seamlessly with GitHub, here, it is considered to be sufficient for publishing documentation in a mirrored fashion alongside an extant GitHub code repository. The documentation of a given source code repository, as stated, should mirror the source code structure, i.e., its directory structure, where the things documented are specified with business logic or project goals, so that things not relevant for documentation, e.g., private methods in public APIs, are not documented. This is the most open aspect of ACDC, and varies primarily with the specific use case of the system.

5. ANALYSIS

The analysis of ACDC is framed as a simple survey study, where the sampling is non-probabilistic and accidental [51]. Four software practitioners, each with a minimum of five years' experience in professional, industry software development, acted as the population for the survey. The breakdown of respondent characteristics can be seen in Table 5. Respondents were asked to rate ACDC-generated documentation for a complex .NET method taken from the Roslyn compiler's source code. ACDC-generated documentation for this method derived from both GPT-3.5 and GPT-4 were both provided, to compare the efficacy of GPT-3.5 and GPT-4. The context provided to ACDC in the document request prompt for this method included its containing class, the usings of its class and its namespace. The survey of these four individuals in particular provides an uninformed basis in evaluating the efficacy of ACDC in documenting code, as none of the four respondents have any significant experience in C#/.NET, and instead work with other languages altogether. The rationale behind this is that if experienced software engineers, without any foreknowledge of the language and source code being reviewed, can digest the provided documentation, then the documentation must possess some merit. To balance these relatively uninformed opinions, a fifth respondent with over 20 years' experience in C#/.NET programming was also asked to evaluate the documented method. The method chosen was the following, taken directly from the GitHub repository of the Roslyn compiler [42]:

- roslyn/src/Analyzers/CSharp/Analyzers/AddAccessibilityModifiers/CSharpAddAccessibilityModifiers.cs
  - public bool ShouldUpdateAccessibilityModifier

The full source of this method can be found in Appendix A. This method is a part of the code analysis namespace of the Roslyn API and contains logic on whether or not an accessibility modifier is to be updated on a given member, where a member refers to a member of a type in C#/.NET. This includes fields, methods, properties, events, and nested types. In other words, a member is a language construct that is a part of a type, e.g., as the fields and methods in some class, MyClass, are all members of the MyClass type.

5.1 Survey Method

Five survey questions were to be answered on a five-point Likert scale, where the range increments in positive attitude, i.e., 1 is poor, 2 is decent, 3 is good, 4 is great and 5 is exceptional. These questions are given below.

<u>Question 1: Comprehensibility</u>
How well do you understand the functionality of the method from the documentation?

<u>Question 2: Accuracy</u>
How accurately does the documentation describe the functionality and use cases of the method?

<u>Question 3: Completeness</u>
To what extent does the documentation include all details (parameters, return values, exceptions, etc.) of the method?

<u>Question 4: Usefulness</u>
How helpful were the provided code examples in understanding how to use the method?

<u>Question 5: Preference</u>
How would you rate the generated documentation compared to traditional, human-written documentation for understanding and implementing the methods?

Table 5. Survey respondent characteristics.

Respondent no.	1	2	3	4
Job/Position	ML engineer	Systems engineer	Software engineer	Software engineer
Education level	Ph.D. student	Ph.D. student	M.S.	Ph.D. student
Gender	M	M	F	M
Age	26	26	23	25

Table 6. Respondent answers to questions about ACDC GPT-3.5 generated documentation (5-point Likert scale).

	Q1: Comprehensibility	**Q2: Accuracy**	**Q3: Completeness**	**Q4: Usefulness**	**Q5: Preference**
R1: ML engineer	3	4	3	2	4
R2: systems engineer	2	2	4	4	4
R3: software engineer	4	3	3	2	3
R4: software engineer	2	2	2	2	3
Average	*2.75*	*2.75*	*3.00*	*2.50*	*3.50*

Table 7. Respondent answers to questions about ACDC GPT-4 generated documentation (5-point Likert scale).

	Q1: Comprehensibility	**Q2: Accuracy**	**Q3: Completeness**	**Q4: Usefulness**	**Q5: Preference**
R1: ML engineer	2	3	3	2	3
R2: systems engineer	2	3	4	4	5
R3: software engineer	2	4	3	3	3
R4: software engineer	2	2	2	2	3
Average	*2.00*	*2.75*	*3.00*	*2.75*	*3.50*

Each respondent was first provided with the source code, and then the documentation was displayed. Each respondent had the ability to open the source code alongside the documentation, if desired. The questions were asked orally with the code documentation open in front of each respondent, individually, with no respondent having any awareness of the other respondents. As specified in the documentation request prompt (see Table 3), the documentation was generated in Markdown format, for ease of viewing. To determine the efficacy of GPT-3.5 and GPT-4, the ACDC-generated documentation using each model were given at once, with the GPT-3.5 documentation on the left, and the GPT-4 documentation on the right, of the same computer screen. The full copy of each of these GPT-generated documentation texts can be found in Appendix B of this paper. With each Question, the respondents usually, without being prompted, complemented their numerical response with a more freeform answer, indicating for what reason(s) they stated their answer. These natural language answers were recorded and some of the more interesting ones are represented in this section.

5.2 Survey Implementation and Analysis

Leveraging the survey method elaborated on in the previous section, the respondents were surveyed, one after another, in separate locations and at separate times. Responses for the GPT-3.5 and GPT-4 documentation are given in Table 6 and Table 7, respectively. Answers to questions varied, except for the questions on accuracy and completeness, where the averages for these two questions did not change. This is possibly because the respondents were not experienced with C#, and simply chose middling-high values to proceed with the survey. However, the systems engineer noted that, in answering the completeness question, he noticed the tables for the two versions of documentation (GPT-3.5 and GPT-4) contained the *out* keyword in different columns, indicating possibly erroneous documentation. The average preference answer did not change, indicating that the respondents may only lightly prefer GPT-generated documentation to human-written documentation. In particular, the systems engineer, who spent the most time of all respondents reading the method body and the documentation, stated, *"I like this documentation because it's more than I usually get"*, implying that it is his belief that human-written documentation is often insufficient for its task. The question on usefulness saw no significant change between documentation versions.

The question on comprehensibility provided the most interesting result, as every respondent categorically rated the documentation from GPT-4 as less comprehensible than that derived from GPT-3.5, because of the verbosity of the former. All of the respondents said something to the effect of *"The GPT-4 documentation is better but it is obviously longer too".* This is an interesting observation, and it opens discussion on the optimal length of documentation for a given method. Moreover, looking at the generated documentation (see Appendix B), the GPT-4 version is lengthier as it includes "reasoning", because the method documented is not easily exemplified in code without providing much more context to ACDC, and the GPT-4 documentation states this literally, along with a disclaimer before its examples. GPT-3.5 did not do this, and directly asserted examples without justification. One respondent asserted that, while the GPT-4 documentation was better, it took a significant amount of time to actually parse its contents as opposed to the documentation from GPT-3.5, and as such, was no more comprehensible than the documentation from GPT-3.5.

Commenting about the length of the documentation from GPT-4, one respondent, the ML engineer, said: *"Looking at [the GPT-4 documentation], I definitely feel more informed than the [GPT-3.5] documentation. But there's a problem because I feel overwhelmed. There's too much to read now, and now it's like, 'Oh… now I have to read all this crap.' So I think you have a problem, you have to bridge that human factors gap of overwhelming me with documentation."* This is a very insightful comment, and the respondent even went so far as to provide a suggestion, as he was experienced with Markdown. He suggested wrapping the Examples section of the documentation in a drop-down element, which in Markdown is done with the <details> tag. GPT-3.5 and GPT-4 are capable of generating Markdown text with <details> tags. This would allow the reader to

selectively inform themselves when encountering the documentation for a given method, where they may not be desirous of overloading themselves with examples immediately.

The same respondent also commented on the usefulness of the table delineating the components of the documented method. He said, *"I don't think the table helps me much. I can just look at the method to see this stuff. Maybe if you can tell me, in the table, how the parameters are actually used in the method, it can be more useful to me."* This observation is important and could, if desired, be addressed rather simply, as an adjustment to the output documentation can be had with an adjustment to the documentation request prompt (see Table 3).

With discussions ensuing about the use of GPT for documentation, one participant, half-jokingly, asked: *"Guess we'll be out of jobs soon, huh?"* Although somewhat humorous, this remark is poignant, and opens discussion on the sociological and praxeological impacts of large-scale generative AI. In March of 2023, researchers at OpenAI released a paper that, among other insights, predicted that 80% of workers will have 10% of their tasks impacted by LLMs, and 19% of workers will see *over half* of their job-related tasks influenced by LLMs [52]. This is not insignificant, and should be considered with gravity, given the honest responses of those surveyed here.

Supplementing this survey was the opinion of a 20+ year expert in C# programming. His responses are indicated below:

- GPT-3.5: *"The conciseness of the summarization is excellent. The examples are simple and show "use" only. The Description is technically dense and assumes prior knowledge. For me this is sufficient and good."*
- GPT-4: *"The answer is longer but not really more complete… I like the grid-like listing of parameters. The 3 paragraphs which lead off the examples seem somewhat snooty. Being told the method 'can be quite complex since it involves using the Roslyn compiler's services' could reflect high-mindedness in the AI training set. The additional detail of the examples is not useful [in my opinion] because it's a simple function which is adequately described without it."*

So, for both the uninformed respondents and the C# expert, the documentation generated from GPT-3.5 was categorically preferred over that from GPT-4, because of the directness and simplicity of the documentation. This is a useful insight, as it may direct future work to consider the faster, cheaper GPT-3.5 over the flagship GPT-4; and, it may provide an empirical incentive for those using GPT-4 to sufficiently constrain its output with more restrictive prompts, to only generate what is needed, and nothing superfluous.

A final aspect to consider is the depth of treatment of GPT-4's documentation. Without being explicitly told so, GPT-4 referenced the Roslyn API as an outside resource to the reader at the end of its documentation, and even linked to the Roslyn SDK homepage (see Appendix B). This is likely a behavior that will need to be constrained if the ACDC system is to be used across an entire codebase, as it is probably not desirable for firms working with proprietary code to have their documentation contain unchecked links to outside, possibly unverified, resources. The prompt engineering techniques discussed in Table may be of use. In any case, the documentation request prompt (see Table 3) will have to be modified and refined through much more experimentation, to eventually arrive at the optimal solution. Doubtless, this optimal solution will vary given diverse business needs and project goals for users of ACDC, which is why the ACDC prompting solution is built to be hot-swappable, where different prompt templates can be substituted with minimal effort.

6. Related Work

In 2018, Hu et al. formulate code summarization as a machine translation problem and use a sequence-to-sequence (seq2seq) language model in combination with source code ASTs to automatically generate code comments [53]. The encoder and decoder in their seq2seq language model were both Long Short-Term Memory (LSTM) units, which improve upon the RNN by introducing the memory cell.

In 2022, Khan and Uddin demonstrated GPT-3 to be very effective at generating simplistic software documentation for a variety of languages, e.g., Java, Python, PHP, JavaScript, Ruby and Go, by treating it as a one-shot learning problem, where one example of documentation is provided with each prompt [44]. Although their work has no integration with a CI/CD pipeline or a large codebase, their observations are useful. The authors recommend keeping the model's temperature value in the low range, at 0.2, which is also adopted here, because it is not desirable to produce wildly unpredictable outputs; temperature values were experimented with, but it was found that 0.2 was preferable. Their evaluations are of interest, because it was found that, by simply giving GPT-3 a single example along with the prompt (1-shot learning), the model performed, on average, twice as well as when prompted with no example in the prompt (0-shot learning). This was done primarily to "force" GPT-3 to adhere to the expected documentation format, a technique mirrored here. It should be noted that, in reality, this is not 1-shot learning, as LLMs are not learning tasks during active prompting, rather, they are just performing task location in their learned task space [35]. But, for the purpose of illustration, this "1-shot learning" idea is a useful prompt engineering technique. Khan and Uddin have also used GPT-3 to produce code examples from documentation [54].

All the related works only present simple facets of ACD, e.g., method documentation. These techniques are, given the power of GPT-3.5 and GPT-4, relatively trivial in implementation, and there has not been proposed a full-suite, end-to-end,

real-time system for ACD that does not simply assist the developer with documentation, but completes it for him and automatically creates pull requests with a source code repository. Moreover, none of the surveyed techniques consider anything beyond source code methods or classes: these are documented atomically, without consideration of the greater context in which these elements are placed, e.g., in project hierarchies, with respect to its invocations, dependence/coupling on other methods/classes, etc. This paper contains the proposal and analysis of such an end-to-end system, ACDC, differentiating itself from other systems because of its unique framing of ACD as a CI/CD task integrated with GitHub, completely removing the need for manual documentation of code.

7. Discussion, Limitations and Future Work

In this section is provided a discussion about the overall placement of ACDC in currently rapid zeitgeist of generative AI, mid-2023. The current state of GPT-3.5 and GPT-4 are discussed, as well as the future development of OpenAI's flagship models and the impact on ACDC. Extensions of ACDC to other areas of the SDLC, in addition to future considerations for the proper completion of ACDC, are also explored here.

7.1 GPT-4

The use of GPT-4 in evaluations was limited, because of its lack of speed when compared with GPT-3.5. Also, GPT-4 is significantly more expensive than GPT-3.5, with GPT-4 (8k context width) costing $0.03/1k tokens and $0.06/1k tokens per prompt and completion, respectively; in comparison, GPT-3.5 (4k context width) costs $0.002/1k tokens per chat [55]. Given a token length of 1,000, this means that using GPT-4 is 45x more expensive than GPT-3.5 (0.03 + 0.06 = 0.09; 0.09 / 0.002 = 45). Given this massive disparity in pricing, where GPT-3.5 is, for most evaluations performed, exceptional at its employed task, GPT-4 was not seen as a prudent choice for the LLM incorporated with the ACDC system. Additionally, the June 13th, 2023 version of GPT-3.5 experienced a 25% cost reduction on input tokens (from $0.002 to $0.0015/1k tokens), making it the most efficient model to use [46].

Notwithstanding, a feature of ACDC is its ease of extensibility. Because the automatic documentation generation ultimately relies on invoking the OpenAI API programmatically, where the output is a plaintext string, it is possible that, in the future, the GPT-4 model could be used in place of GPT-3.5, by just modifying the API request. In limited evaluations with GPT-4, it adhered to the requested format and produced output comparable to, if not better, than GPT-3.5, so it remains a distinct and promising possibility.

The analysis section of this work, although very preliminary, demonstrates that expert software practitioners unanimously agree that code documentation generated with GPT-3.5, as opposed to GPT-4, through the ACDC system, was better understood and more useful, although no respondent indicated they preferred GPT-generated documentation over what would be written by a human. This is primarily because GPT-4 generates much lengthier documentation that is overwhelming and confusing for the readers. In future work, it would be prudent to include authentic human-written documentation for the same code, to judge more precisely which documentation is preferred, and why. This would require the location and use of a well-documented code library with active maintainers and verbose documentation to perform the experiment with.

7.2 GPT Evolution and Context Length

It is necessary to consider the context length of the GPT models. In NLP, context length is the number of tokens a model can consider from input text. Practically speaking in terms of the OpenAI API, context length is the shared number of tokens between input and output text. A primary limitation to the advancement of GPT in SDLC automation activities is the context length of the model, as input text, e.g., methods, entire classes, etc., can be quite lengthy, as can output text. Prior to June of 2023, GPT-3.5 had a context length of 4k; in June of 2023, OpenAI released *gpt-3.5-turbo-16k*, which has four times the context length as the previous model [46]. GPT-4 also has a 32k context width version, which would allow eight times the context length of the original GPT-3.5 Chat model. This means that there is a greater potential bandwidth for input text and output responses as developments from OpenAI emerge.

With the same update, users of the new GPT-3.5 and GPT-4 can specify functions, to which GPT will output a JSON object that adheres to the signature of the function. This makes the connection of GPT with other APIs simpler. In July of 2023, OpenAI brought beta access of the Code Interpreter to ChatGPT, which allows GPT to execute Python code and upload files, e.g., source code files and data files, with the ability to read from files for data analysis, plot charts, calculate math functions, e.g., linear regression, run simulations, etc., all within the ChatGPT interface. As of March 2023, OpenAI has granted beta access to ChatGPT plugins, that provide a host of new functionalities to be baked into the ChatGPT experience [56]. Plugins to ChatGPT vary broadly, from travel booking apps to video editing frameworks to diagram editing programs, and many others.

These contemporary updates to GPT are mentioned here because it lends credence to the thinking that large-scale generative AI, e.g., GPT-3.5 and GPT-4, will be tightly woven in with traditional software development practices in the near future. The work presented in this paper with the proposed ACDC system is just one concrete idea in this rapidly accelerating field.

In any case, one facet of ACDC not fully developed is the assignment of a *max_tokens* value for API requests. As discussed previously (see Figure 3), the calculation of a *max_tokens* value is most likely dependent on business needs or project goals, to allow for full documentation output, while disallowing too-long documentation, that would prove both too costly and too overwhelming for documentation readers. The initial calculation is to tokenize the input prompt and subtract this token length from the context width of the used model; but this leaves a *max_tokens* value at its actual maximum, which is likely undesirable and possibly costly. So, a potential solution is some form of dynamic or adaptive scaling, where this actual maximum value is scaled down, based on an analysis of the input text and the desired output, e.g., brief documentation, verbose documentation, etc. Source code measurement and analysis are longstanding fields of study rife with various tools, e.g., SonarQube, methods and metrics for doing so [57]. This remains an area of future work.

7.3 Extension to other SDLC Activities

Because the ACDC system relies primarily on the generation of properly contextualized code prompts derived from ASTs, it is quite possible to generalize the underlying ACDC workflow to subsume various activities of the SDLC, principally those that can be considered "roundup" or "review" activities, e.g., code conciseness checking, code formatting, cohesion improvement, general refactoring, and so on. Moreover, because of the advanced insight of the GPT-3.5 and GPT-4 LLMs, which possess billions of parameters and are trained on immense amounts of code and programming knowledge, other, less obvious SDLC activities can be automated with the ACDC workflow, e.g., improving code security. Additionally, it should be apparent that what is presented in this work is applicable to any domain and development with any programming language, as GPT-3.5 and GPT-4 are well capable at resolving prompts for a plethora of programming languages, not simply C#/.NET.

The general workflow for a GPT-enabled CI/CD system can be broken down into a series of steps:

1. Determine the task, e.g., documentation
2. Generate semantic code representation of relevant source code files, e.g., ASTs
3. Select code context(s) of relevance, e.g., invocation hierarchies
4. Experiment with relevant prompts to complete the desired task, using techniques described here
5. Store prompt as a template and make the CI/CD task an automatable API call
6. Get response and format, store, push and make pull request as needed, e.g., with Octokit

Automatic code documentation provides a concrete, straightforward task for evaluating the potential future of the software development discipline being integrated with large-scale generative AI. In this work, the potential is apparent and promising, as indicated by the preliminary survey performed on a number of software practitioners.

7.4 Limitations and Future Work

Although the preliminary survey presented as part of this work indicated that GPT-generated documentation for complex C# code is promising, an issue encountered is the verbosity of the generated documentation, especially that derived from GPT-4. Overly verbose documentation was a common complaint for the survey respondents. Plans to remediate this include a more precise calculation of the *max_tokens* value for limiting the response length of GPT API requests, in addition to more restrictive prompting techniques.

If generated documentation is not as desired, it is possible, given GPT models with longer context lengths arriving more frequently, to one- or few-shot prompt the used GPT model with example documentation. Taking this thought further, the investigation of GPT model fine-tunes [58], or even local GPT "clones" that can read from the file system, may be worthwhile to generate documentation that better aligns with business needs or project goals. The GPT4All project hosts a collection of open-source chatbots that boast similar performances to GPT-3, all running locally on a CPU [59]. PrivateGPT extends GPT4All and allows the interaction with local documents [60].

The tail-end process of ACDC, namely, appropriately merging generated documentation and creating pull requests for a ReadTheDocs-enabled GitHub repo, is in nascent development. This is a complex undertaking, as it requires deeper consideration of business needs or project goals. For example, consider some class that has two methods to be documented, where one method is private and one is public. For some example organization seeking to document their code, they may have a business rule to only document public functions, or functions callable through their client API. Moreover, once the desired methods are documented, their resolution into a formal document structure should, ideally, reflect the structure of the source code, such that the ReadTheDocs repository mirrors the source code repository, and the documentation is navigable seamlessly.

As demonstrated in this work, ReadTheDocs was chosen as the documentation publication platform because of its easy integration with GitHub; but many other platforms exist (see Table 4) and could be pushed to instead of ReadTheDocs, assuming the documentation request prompt is modified to produce output documentation ingestible by the chosen documentation platform, as the varying platforms employ varying documentation formats.

What has been presented in this work thus far presupposes the existence of only one branch in a given GitHub repository. This is not a realistic position, nor is it tenable with real-world systems, as branches are used for various developmental stages,

e.g., prototyping, testing, deployment, etc. With the foundational capability established by ACDC, future work should investigate syntax tree generation for source code across branches for a given repository. This would provide the user of ACDC with federated source code context that may be integrated into prompts to GPT, potentially allowing for more accurate output.

8. Conclusion

This paper presents a novel, conceptually simple system, Automatic Code Documentation with Compilers, coined ACDC, that utilizes ASTs generated from the Roslyn compiler and GPT-3.5 to produce C# code documentation automatically. The system is integrated with GitHub through Octokit, forming a new CI/CD pipeline, in that every code push triggers an ACDC request for updated documentation, entirely removing the need for the developer to write documentation manually.

A small, preliminary study on the effectiveness of ACDC for generating code documentation for complex C# code provided promising results, where the respondents' average rating for preference of GPT-generated documentation over human-written documentation was "good", above the midpoint on a 5-point Likert scale. GPT-3.5 was determined to be the preferred model, even over the improved GPT-4, because of its significantly lower cost and lack of verbosity in output, which was determined to inhibit documentation understanding for the readers.

It is the authors' intent that this work be placed firmly as a guidepost in the rapidly accelerating stream of research into generative AI and software development automation. What has been presented is a pragmatic system that addresses a pain point in the SDLC for developers, to wit, code documentation, by using GPT to automate the process. Because of the near universal nature of GPT as a knowledge source, and its ability as a programming assistant, the fulfillment of the ACDC system is easily applied to the documentation of source code in any programming language; but doubtless, further experimentation is required to fully validate such a statement. Moreover, the use of a CI/CD system like ACDC is applicable to any facet of the SDLC, particularly those where code is digested and represented in some manner, such as refactoring activities, e.g., formatting, conciseness checking, etc.

The development of the ACDC system has occurred at a precipitous and frenetic time in the history of computation. New developments and applications of large-scale generative AI occur literally with each passing day, in mid-2023. In the months and years to come, there is no doubt that large-scale AI will expand in use. ACDC presents a theoretically grounded system that mitigates some of the stochastic uncertainty of using GPT through ASTs and well-established prompt engineering techniques to automate the process of code documentation for software practitioners.

Acknowledgements

The authors would like to extend their gratitude to Timothy Elvira and Lynn Vonder Haar of Embry-Riddle Aeronautical University, and David Schwartz of University of Illinois at Chicago, for assisting in the formation and execution of the survey.

References

[1] P. Bourque and R. E. Fairley, "Guide to the Software Engineering Body of Knowledge, Version 3.0," *IEEE Computer Society,* 2014.

[2] Y. B. Leau, W. K. Loo, W. Y. Tham and S. F. Tan, "Software development life cycle AGILE vs traditional approaches," *International Conference on Information and Network Technology,* vol. 37, no. 1, pp. 162-167, 2012.

[3] S. Bobrovskis and A. Jurenoks, "A Survey of Continuous Integration, Continuous Delivery and Continuos Deployment," *BIR Workshops,* 2018.

[4] S. A. I. B. S. Arachchi and I. Perera, "Continuous Integration and Continuous Delivery Pipeline Automation for Agile Software Project Management," *2018 Moratuwa Engineering Research Conference (MERCon),* pp. 156-161, 2018.

[5] jenkinsci, "Jenkins Automation Server," Github, [Online]. Available: https://github.com/jenkinsci/jenkins. [Accessed 2023].

[6] travis-ci, "Travis CI," Github, [Online]. Available: https://github.com/travis-ci/travis-ci. [Accessed 2023].

[7] Y. Shmerlin, I. Hadar, D. Kliger and H. Makabee, "To document or not to document? An exploratory study on developers' motivation to document code," *Advanced Information Systems Engineering Workshops: CAiSE 2015 International Workshops,* vol. 27, pp. 100-106, 2015.

[8] P. W. McBurney, S. Jiang, M. Kessentini, N. A. Kraft, A. Armaly, M. W. Mkaouer and C. McMillan, "Towards prioritizing documentation effort," *IEEE Transactions on Software Engineering,* vol. 44, no. 9, pp. 897-913, 2017.

[9] P. W. McBurney, "Automatic Documentation Generation via Source Code Summarization," *2015 IEEEE/ACM 37th IEEE International Conference on Software Engineering,* pp. 903-906, 2015.

[10] H. F. Soares, N. S. R. Alves, T. S. Mendes, M. Mendonça and R. O. Spínola, "Investigating the Link between User Stories and Documentation Debt on Software Projects," *2015 12th International Conference on Information Technology - New Generations,* pp. 385-390, 2015.

[11] Y. Liu and M. Lapata, "Text summarization with pretrained encoders," *arXiv preprint arXiv:1908.08345,* 2019.

[12] B. Wei, L. Ge, X. Xia, Z. Fu and Z. Jin, "Code generation as a dual task of code summarization," *Advances in neural information processing systems,* vol. 32, 2019.

[13] G. Sridhara, E. Hill, D. Muppaneni, L. Pollock and K. Vijay-Shanker, "Towards Automatically Generating Summary Comments for Java Methods∗," *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering - ASE 2010,* pp. 43-52, 2010.

[14] L. Moreno, J. Aponte, G. Sridhara, A. Marcus, L. Pollock and K. Vijay-Shanker, "Automatic Generation of Natural Language," *2013 21st International Conference on Program Comprehension (ICPC),* 2013.

[15] S. Haiduc, J. Aponte, L. Moreno and A. Marcus, "On the Use of Automated Text Summarization Techniques for Summarizing Source Code," *2010 17th Working Conference on Reverse Engineering,* 2010.

[16] P. W. McBurney and C. McMillan, "Automatic documentation generation via source code summarization of method context," *Proceedings of the 22nd International Conference on Program Comprehension,* pp. 279-290, 2014.

[17] E. Hill, "Integrating natural language and program structure information to improve software search and exploration," *University of Delaware,* 2010.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems* , vol. 30, 2017.

[19] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. D. P. Kaplan, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan and R. Child, "Language Models are Few-Shot Learners," *Advances in neural information processing systems,* vol. 33, pp. 1877-1901, 2020.

[20] A. Radford, K. Narasimhan, T. Salimans and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," 2018.

[21] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, "Language Models are Unsupervised Multitask Learners," *OpenAI Blog,* vol. 1, no. 8, p. 9, 2019.

[22] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652,* 2021.

[23] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez and A. Rao, "PaLM: Scaling Language Modeling with Pathways," *arXiv preprint arXiv:2204.02311,* 2022.

[24] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805,* 2018.

[25] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," *arXiv preprint arXiv:1910.13461,* 2019.

[26] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971,* 2023.

[27] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf and G. Sastry, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374,* 2021.

[28] A. Svyatkovskiy, S. K. Deng, S. Fu and N. Sundaresan, "IntelliCode Compose: Code Generation using Transformer," *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering,* pp. 1433-1443, 2020.

[29] Z. Wojciech, "OpenAI Codex," OpenAI, 10 August 2021. [Online]. Available: https://openai.com/blog/openai-codex. [Accessed 2023].

[30] N. Nguyen and S. Nadi, "An empirical evaluation of GitHub copilot's code suggestions," *Proceedings of the 19th International Conference on Mining Software Repositories,* pp. 1-5, 2022.

[31] M. Wermelinger, "Using GitHub Copilot to Solve Simple Programming Problems," *In-Press,* 2023.

[32] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais and Z. M. Jiang, "Github copilot ai pair programmer: Asset or liability?," *Journal of Systems and Software,* 2023.

[33] OpenAI, "GPT-4 Technical Report," *arXiv:2303.08774,* 2023.

[34] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro and Y. Zhang, "Sparks of Artificial General Intelligence: Early experiments with GPT-4," *arXiv preprint arXiv:2303.12712,* 2023.

[35] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems,* pp. 1-7, 2021.

[36] V. Liu and L. B. Chilton, "Design guidelines for prompt engineering text-to-image generative models," *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems,* 2022.

[37] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382,* 2023.

[38] J. White, S. Hays, Q. Fu, J. Spencer-Smith and D. C. Schmidt, "Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," *arXiv preprint arXiv:2303.07839,* 2023.

[39] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, W. Dai, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys,* vol. 55, no. 12, pp. 1-38, 2023.

[40] E. M. Bender, T. Gebru, A. McMillan-Major and S. Shmitchell, "Bender, Emily M., Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? 🦜," *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency,* pp. 610-623, 2021.

[41] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, & Tools, Pearson/Addison Wesley, 2006.

[42] "Roslyn," dotnet, [Online]. Available: https://github.com/dotnet/roslyn. [Accessed May 2023].

[43] B. Wagner, "Explore code with the Roslyn syntax visualizer in Visual Studio," Microsoft, 11 October 2022. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/syntax-visualizer?. [Accessed May 2023].

[44] J. Y. Khan and G. Uddin, "Automatic Code Documentation Generation Using GPT-3," *37th IEEE/ACM International Conference on Automated Software Engineering,* pp. 1-6, 2022.

[45] Y. Yoo, R. J. Boland Jr, K. Lyytinen and A. Majchrzak, "Organizing for innovation in the digitized world," *Organization Science,* vol. 23, no. 5, pp. 1398-1408, 2012.

[46] A. Eleti, J. Harris and L. Kilpatrick, "Function calling and other API updates," OpenAI, 13 June 2023. [Online]. Available: https://openai.com/blog/function-calling-and-other-api-updates. [Accessed 2023].

[47] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, G. Le, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, N. Duan and N. Sundaresan, "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation," *arXiv preprint arXiv:2102.04664,* 2021.

[48] J. Gu, P. Salza and H. C. Gall, "Assemble Foundation Models for Automatic Code Summarization," *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER),* pp. 935-946, 2022.

[49] T. Sanders, "https://github.com/openai/openai-cookbook/blob/main/examples/How_to_count_tokens_with_tiktoken.ipynb," OpenAI, 2023. [Online]. Available: https://github.com/openai/openai-cookbook/blob/main/examples/How_to_count_tokens_with_tiktoken.ipynb. [Accessed 2023].

[50] octokit, "octokit.net," GitHub, [Online]. Available: https://github.com/octokit/octokit.net. [Accessed 2023].

[51] J. Linaker, S. M. Sulaman, M. Höst and R. M. de Mello, "Guidelines for conducting surveys in software engineering v. 1.1," *Lund University,* vol. 50, 2015.

[52] T. Eloundou, S. Manning, P. Mishkin and D. Rock, "GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models," *arXiv:2303.10130v4,* 2023.

[53] X. Hu, G. Li, X. Xia, D. Lo and Z. Jin, "Deep code comment generation," *Proceedings of the 26th conference on program comprehension,* pp. 200-210, 2018.

[54] J. Y. Khan and G. Uddin, "Combining Contexts from Multiple Sources for Documentation-Specific Code Example Generation," *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER),* pp. 683-687, 2023.

[55] "Pricing," OpenAI, 2023. [Online]. Available: https://openai.com/pricing. [Accessed May 2023].

[56] OpenAI, "ChatGPT plugins," OpenAI, 23 March 2023. [Online]. Available: https://openai.com/blog/chatgpt-plugins. [Accessed 2023].

[57] A. S. Nuñez-Varela, H. G. Pérez-Gonzalez, F. E. Martínez-Perez and C. Soubervielle-Montalvo, "Source code metrics: A systematic mapping study," *Journal of Systems and Software,* vol. 128, pp. 164-197, 2017.

[58] OpenAI, "Fine-tuning," OpenAI, [Online]. Available: https://platform.openai.com/docs/guides/fine-tuning. [Accessed 2023].

[59] Y. Anand, Z. Nussbaum, B. Duderstadt, B. Schmidt and A. Mulyar, "GPT4All: Training an Assistant-style Chatbot with Large Scale Data Distillation from GPT-3.5-Turbo," 2023. [Online]. Available: https://github.com/nomic-ai/gpt4all. [Accessed 2023].

[60] I. Martinez, "privateGPT," GitHub, 2023. [Online]. Available: https://github.com/imartinez/privateGPT. [Accessed 2023].

APPENDIX A: THE ROSLYN SOURCE CODE EVALUATED IN THE SURVEY OF ACDC

```
using Microsoft.CodeAnalysis.AddAccessibilityModifiers;
using Microsoft.CodeAnalysis.CodeStyle;
using Microsoft.CodeAnalysis.CSharp.Extensions;
using Microsoft.CodeAnalysis.CSharp.Syntax;
using Microsoft.CodeAnalysis.LanguageService;
using Microsoft.CodeAnalysis.Shared.Extensions;

namespace Microsoft.CodeAnalysis.CSharp.AddAccessibilityModifiers {
    internal class CSharpAddAccessibilityModifiers : AbstractAddAccessibilityModifiers<MemberDeclarationSyntax> {
        public static readonly CSharpAddAccessibilityModifiers Instance = new();

        protected CSharpAddAccessibilityModifiers() {
        }

        public override bool ShouldUpdateAccessibilityModifier(
            IAccessibilityFacts accessibilityFacts,
            MemberDeclarationSyntax member,
            AccessibilityModifiersRequired option,
            out SyntaxToken name,
            out bool modifierAdded)
        {
            modifierAdded = false;

            name = member.GetNameToken();
            if (name.Kind() == SyntaxKind.None)
                return false;

            if (!accessibilityFacts.CanHaveAccessibility(member))
                return false;

            var accessibility = accessibilityFacts.GetAccessibility(member);

            var isOmit = option == AccessibilityModifiersRequired.OmitIfDefault;
            modifierAdded = !isOmit;

            if (isOmit) {
                if (accessibility == Accessibility.NotApplicable)
                    return false;

                var parentKind = member.GetRequiredParent().Kind();
                switch (parentKind) {
                    case SyntaxKind.CompilationUnit:
                    case SyntaxKind.FileScopedNamespaceDeclaration:
                    case SyntaxKind.NamespaceDeclaration: {
                        if (accessibility != Accessibility.Internal)
                            return false;
                    }

                        break;

                    case SyntaxKind.ClassDeclaration:
                    case SyntaxKind.RecordDeclaration:
                    case SyntaxKind.StructDeclaration:
                    case SyntaxKind.RecordStructDeclaration: {
                        if (accessibility != Accessibility.Private)
                            return false;
                    }

                        break;

                    default:
                        return false;
                }
            }
            else {
                if (accessibility != Accessibility.NotApplicable)
                    return false;
            }

            return true;
        }
    }
}
```

APPENDIX B: THE GPT-GENERATED DOCUMENTATION

Markdown documentation generated with GPT-3.5 for the *ShouldUpdateAccessibilityModifier* method in Appendix A:

```
# CSharpAddAccessibilityModifiers

## Parameters
| Parameter | Type | Description |
| --- | --- | --- |
| `accessibilityFacts` | `IAccessibilityFacts` | The accessibility facts service. |
| `member` | `MemberDeclarationSyntax` | The member declaration syntax node. |
| `option` | `AccessibilityModifiersRequired` | The option for required accessibility modifiers. |
| `name` | `out SyntaxToken` | The name of the member declaration. |
| `modifierAdded` | `out bool` | Indicates whether a modifier was added. |

## Description
This function determines whether an accessibility modifier should be updated for a member declaration syntax node, based on the provided accessibility facts, option for required accessibility modifiers, and the current accessibility modifier of the member. It returns `true` if the accessibility modifier should be updated, and provides the name of the member declaration and a flag indicating whether a modifier was added as output parameters.

## Examples
```csharp
// Example 1: Update accessibility modifier
bool shouldUpdate = Instance.ShouldUpdateAccessibilityModifier(
    accessibilityFacts,
    member,
    AccessibilityModifiersRequired.OmitIfDefault,
    out SyntaxToken name,
    out bool modifierAdded);
```

```csharp
// Example 2: Check if accessibility modifier should be updated
bool shouldUpdate = Instance.ShouldUpdateAccessibilityModifier(
    accessibilityFacts,
    member,
    AccessibilityModifiersRequired.Always,
    out SyntaxToken name,
    out bool modifierAdded);
```
```

Markdown documentation generated with GPT-4 for the *ShouldUpdateAccessibilityModifier* method in Appendix A:

```
# CSharpAddAccessibilityModifiers
The `CSharpAddAccessibilityModifiers` class is a part of the `Microsoft.CodeAnalysis.CSharp.AddAccessibilityModifiers` namespace and
it inherits from `AbstractAddAccessibilityModifiers<MemberDeclarationSyntax>`.

## Description
The `CSharpAddAccessibilityModifiers` class contains the functionality to decide whether an accessibility modifier should be added
or updated for a given member. It is specifically used to check the conditions based on the accessibility of the member and the
options provided.

### ShouldUpdateAccessibilityModifier
Determines whether an accessibility modifier should be updated or not for a member declaration.

| Parameter | Type | Description |
|---|---|---|
| `accessibilityFacts` | IAccessibilityFacts | An instance providing accessibility information |
| `member` | MemberDeclarationSyntax | The member for which to determine if the accessibility modifier should be updated |
| `option` | AccessibilityModifiersRequired | The option to consider for updating the accessibility modifier |
| `name` (out) | SyntaxToken | The name token of the member |
| `modifierAdded` (out) | bool | Indicates whether the modifier was added |

## Examples
Examples for this method can be quite complex since it involves using the Roslyn compiler's services and dealing with the internal
mechanism of the code analysis. However, let's consider a simple scenario where we would like to check if a method's or field's
accessibility modifier should be updated or not.

The examples are illustrative and simplified. In a real use case, you would need to get instances of IAccessibilityFacts and
MemberDeclarationSyntax from the Roslyn compiler's services and the analyzed source code.

The following examples illustrate how to use `ShouldUpdateAccessibilityModifier` method to determine if an accessibility modifier
should be updated or not for a given member. In these examples, we are considering a public method and a private field in a class.
The method returns if the modifier should be updated or not, and outputs the name of the member and if the modifier was added or
not.

```csharp
using Microsoft.CodeAnalysis.CSharp.Syntax;
using Microsoft.CodeAnalysis.AddAccessibilityModifiers;

// Assume `accessibilityFacts` is an instance of a class implementing IAccessibilityFacts
IAccessibilityFacts accessibilityFacts = ... ;

// Example 1: Analyzing a public method in a class
MethodDeclarationSyntax method = SyntaxFactory.MethodDeclaration(SyntaxFactory.ParseTypeName("void"), "MyMethod")
    .AddModifiers(SyntaxFactory.Token(SyntaxKind.PublicKeyword));

SyntaxToken name;
bool modifierAdded;
bool shouldUpdate = CSharpAddAccessibilityModifiers.Instance.ShouldUpdateAccessibilityModifier(
    accessibilityFacts,
    method,
    AccessibilityModifiersRequired.OmitIfDefault,
    out name,
    out modifierAdded);

Console.WriteLine($"Should update: {shouldUpdate}, Name: {name}, Modifier Added: {modifierAdded}");

// Example 2: Analyzing a private field in a class
FieldDeclarationSyntax field = SyntaxFactory.FieldDeclaration(
    SyntaxFactory.VariableDeclaration(SyntaxFactory.ParseTypeName("int"))
    .AddVariables(SyntaxFactory.VariableDeclarator("myField")))
    .AddModifiers(SyntaxFactory.Token(SyntaxKind.PrivateKeyword));

shouldUpdate = CSharpAddAccessibilityModifiers.Instance.ShouldUpdateAccessibilityModifier(
    accessibilityFacts,
    field,
    AccessibilityModifiersRequired.OmitIfDefault,
    out name,
    out modifierAdded);

Console.WriteLine($"Should update: {shouldUpdate}, Name: {name}, Modifier Added: {modifierAdded}");
```

In these examples, we create instances of MethodDeclarationSyntax and FieldDeclarationSyntax (which both are subclasses of
MemberDeclarationSyntax) to represent a method and a field in a class, and then we use the ShouldUpdateAccessibilityModifier method
to check if their accessibility modifiers should be updated according to the AccessibilityModifiersRequired.OmitIfDefault rule. The
method returns if the modifier should be updated or not, and outputs the name of the member and if the modifier was added or not.

Remember, in a real-world scenario, the accessibilityFacts instance and member would be obtained through the Roslyn API from the
source code being analyzed.

For more detailed information on how to use Roslyn and work with its syntax trees, please refer to the official Roslyn documentation:
https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/
```